

Criando uma anotação personalizada em Java

Última atualização: 8 de janeiro de 2024



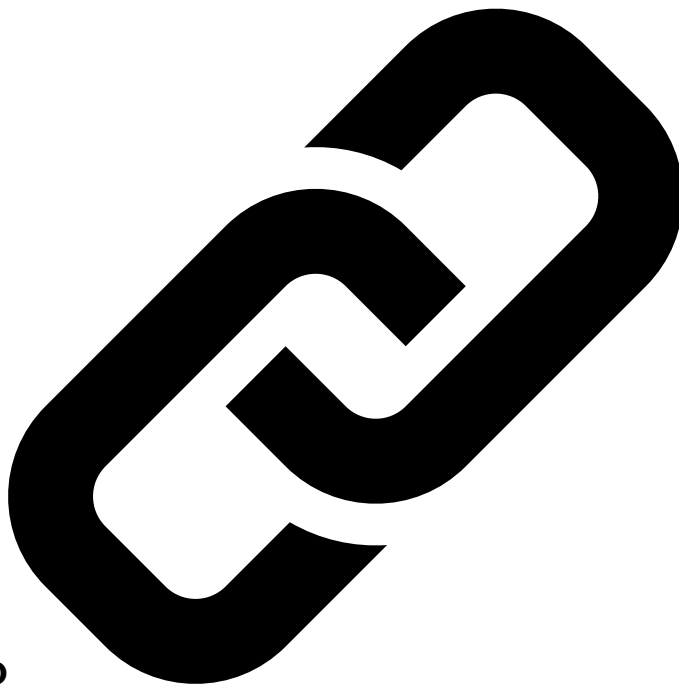
Escrito por:
desânimo



Revisado por:
Tom Homberg

Núcleo Java

Anotações
Java



1. Introdução

Anotações Java são um mecanismo para adicionar informações de metadados ao nosso código-fonte. Elas são uma parte poderosa do Java que foi adicionada no JDK5. Anotações oferecem uma alternativa ao uso de descritores XML e interfaces de marcadores.

Embora possamos anexá-las a pacotes, classes, interfaces, métodos e campos, as anotações por si só não têm efeito na execução de um programa.

Neste tutorial, vamos nos concentrar em como criar e processar anotações personalizadas. Podemos ler mais sobre anotações em [nosso artigo sobre noções básicas de anotação](#).

Leitura adicional:

Classes abstratas em Java

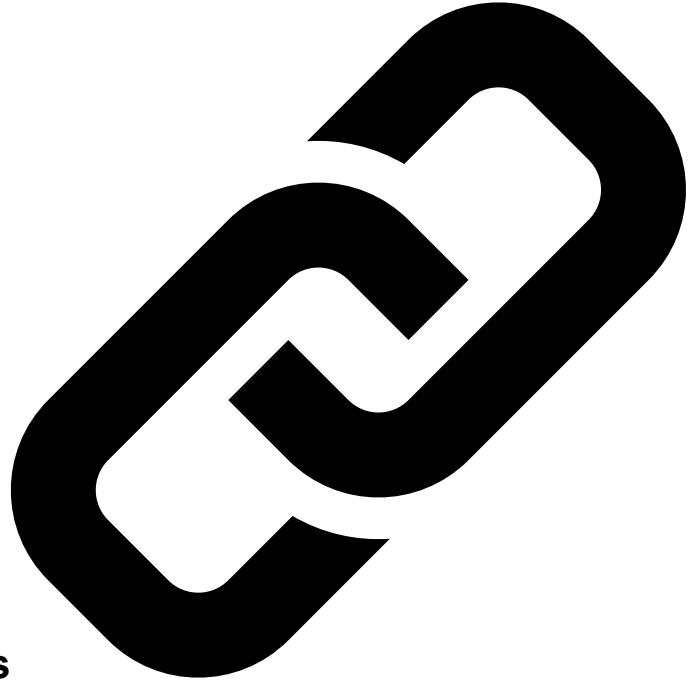
Aprenda como e quando usar classes abstratas como parte de uma hierarquia de classes em Java.

[Leia mais →](#)

Interfaces de marcadores em Java

Aprenda sobre interfaces de marcadores Java e como elas se comparam a interfaces e anotações típicas

[Leia mais →](#)



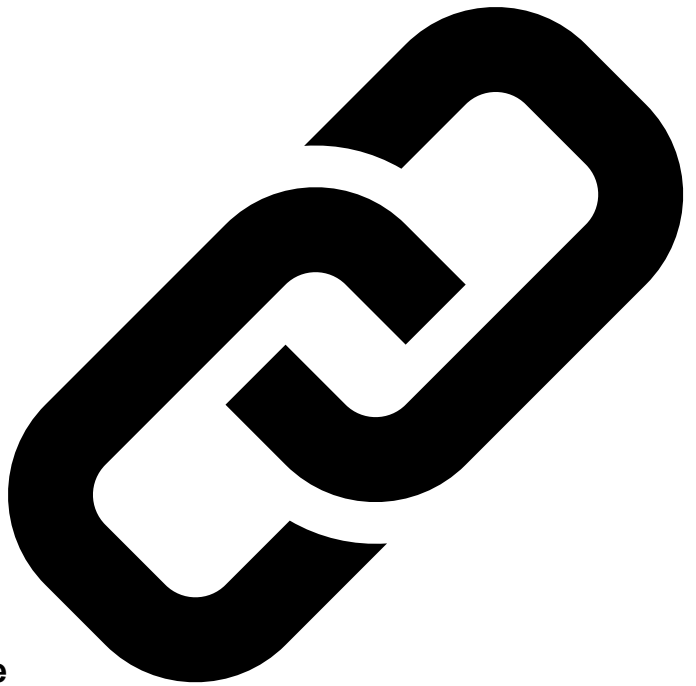
2. Criando Anotações Personalizadas

Vamos criar três anotações personalizadas com o objetivo de serializar um objeto em uma string JSON.

Usaremos o primeiro no nível de classe, para indicar ao compilador que nosso objeto pode ser serializado. Então, aplicaremos o segundo aos campos que queremos incluir na string JSON.



Por fim, usaremos a terceira anotação no nível do método para especificar o método que usaremos para inicializar nosso objeto.



2.1. Exemplo de anotação de nível de classe

O primeiro passo para criar uma anotação personalizada é **declará-la usando a palavra-chave `@interface`** :

```
public @interface JsonSerializable {  
}
```

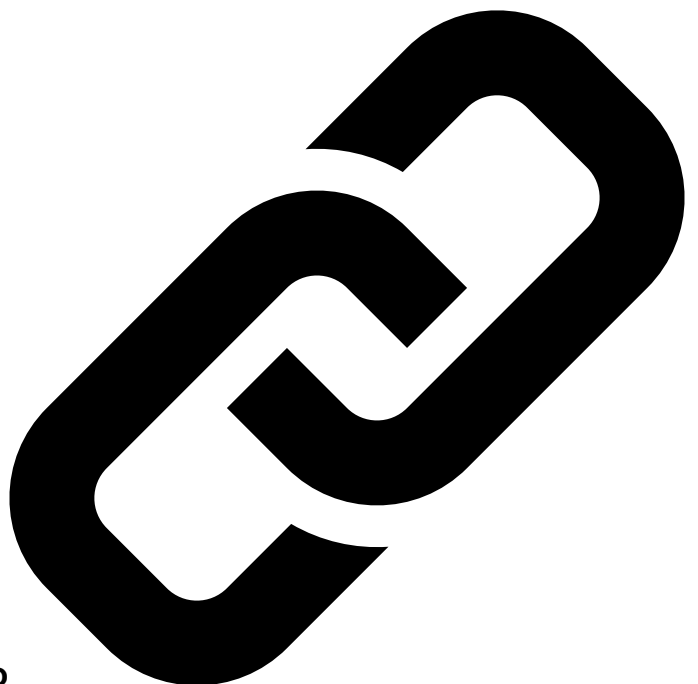


O próximo passo é **adicionar meta-anotações para especificar o escopo e o alvo** da nossa anotação personalizada:

```
@Retention(RetentionPolicy.RUNTIME)  
@Target(ElementType.TYPE)  
public @interface JsonSerializable {  
}
```

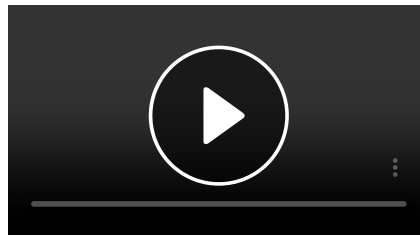


Como podemos ver, nossa primeira anotação **tem visibilidade de tempo de execução, e podemos aplicá-la a tipos (classes)** . Além disso, ela não tem métodos, e assim serve como um marcador simples para marcar classes que podem ser serializadas em JSON.



2.2. Exemplo de anotação em nível de campo

Da mesma forma, criamos nossa segunda anotação para marcar os campos que vamos incluir no JSON gerado:



Patrocinado pela SABIC

Como estamos ajudando esses pezinhos a deixarem uma pegada...

Nossa colaboração com a Drylock e o Grupo Plastik está ajudando a criar uma solução para que as fraldas gerem menos resíduos, por meio da utilização dos polímeros circulares certificad...

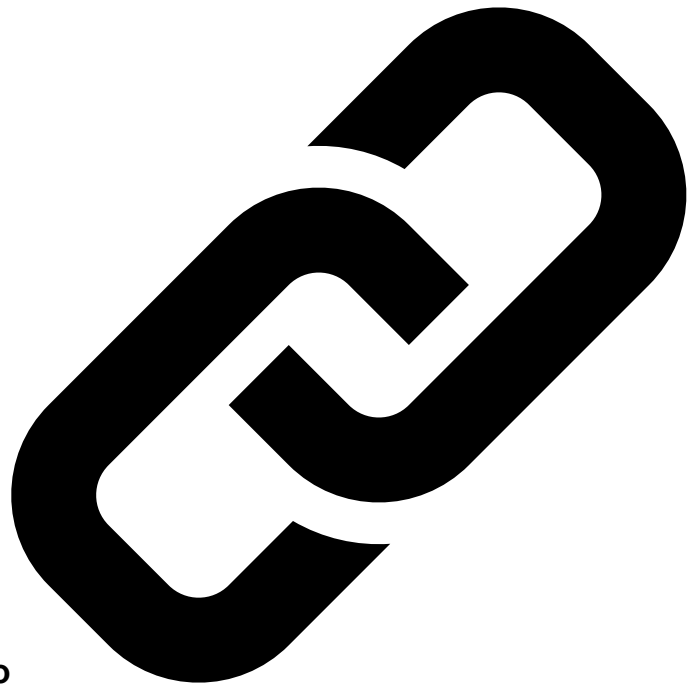
[Saber mais](#)

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface JsonElement {
    public String key() default "";
}
```



A anotação declara um parâmetro String com o nome “key” e uma string vazia como valor padrão.

Ao criar anotações personalizadas com métodos, devemos estar cientes de que esses **métodos não devem ter parâmetros e não podem lançar uma exceção**. Além disso, **os tipos de retorno são restritos a primitivos, String, Class, enums, anotações e arrays desses tipos, e o valor padrão não pode ser null**.



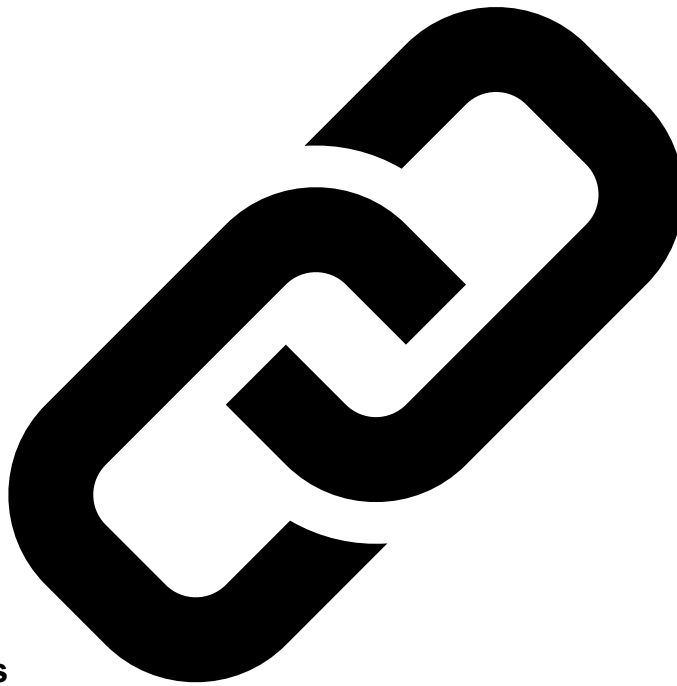
2.3. Exemplo de Anotação de Nível de Método

Vamos imaginar que antes de serializar um objeto para uma string JSON, queremos executar algum método para inicializar um objeto. Por esse motivo, vamos criar uma anotação para marcar esse método:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Init {
}
```

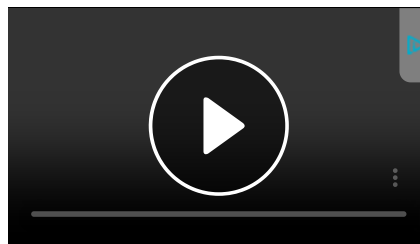


Declaramos uma anotação pública com visibilidade em tempo de execução que podemos aplicar aos métodos de nossas classes.



2.4. Aplicando Anotações

Agora vamos ver como podemos usar nossas anotações personalizadas. Por exemplo, vamos imaginar que temos um objeto do tipo *Person* que queremos serializar em uma string JSON. Esse tipo tem um método que coloca a primeira letra do primeiro e último nome em maiúscula. Vamos querer chamar esse método antes de serializar o objeto:



Patrocinado pela SABIC

Como estamos ajudando esses pezinhos a deixarem uma pegada...

Nossa colaboração com a Drylock e o Grupo Plastik está ajudando a criar uma solução para que as fraldas gerem menos resíduos, por meio da utilização dos polímeros circulares certificad...

Saber mais

```
@JsonSerializable
public class Person {

    @JsonElement
    private String firstName;

    @JsonElement
    private String lastName;

    @JsonElement(key = "personAge")
    private String age;

    private String address;

    @Init
    private void initNames() {
        this.firstName = this.firstName.substring(0, 1).toUpperCase()
            + this.firstName.substring(1);
        this.lastName = this.lastName.substring(0, 1).toUpperCase()
            + this.lastName.substring(1);
    }
}
```

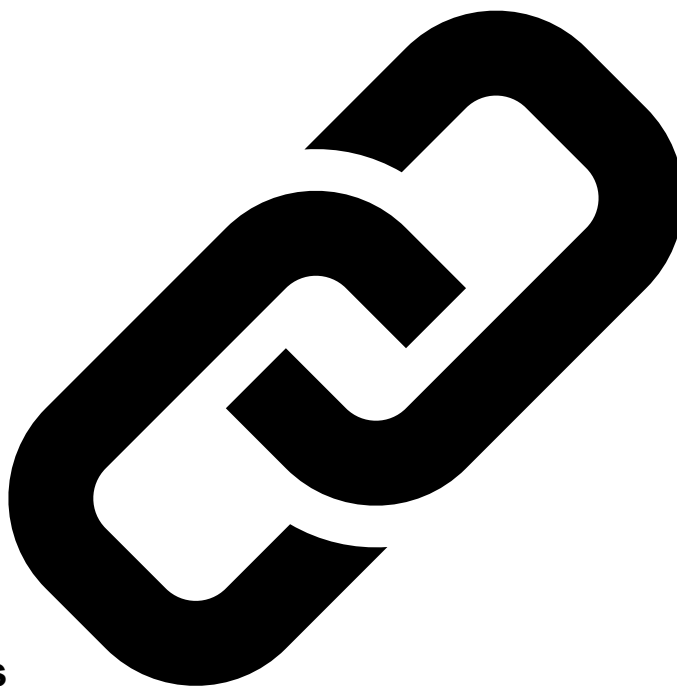


```
// Standard getters and setters  
}
```

Ao usar nossas anotações personalizadas, estamos indicando que podemos serializar um objeto *Person* para uma string JSON. Além disso, a saída deve conter apenas os campos *firstName*, *lastName* e *age* desse objeto. Além disso, queremos que o método *initNames()* seja chamado antes da serialização.

Ao definir o parâmetro *-chave* da anotação *@JsonElement* como "personAge", estamos indicando que usaremos esse nome como o identificador do campo na saída JSON.

Para fins de demonstração, tornamos *initNames()* privado, então não podemos inicializar nosso objeto chamando-o manualmente, e nossos construtores também não o estão usando.



3. Processando Anotações

Até agora, vimos como criar anotações personalizadas e como usá-las para decorar a classe *Person*. Agora, veremos como tirar vantagem delas usando a API Reflection do Java.

O primeiro passo será verificar se nosso objeto é *nulo* ou não, bem como se seu tipo possui a anotação *@JsonSerializable* ou não:

```
private void checkIfSerializable(Object object) {  
    if (Objects.isNull(object)) {  
        throw new JSONException("The object to serialize is null");  
    }  
  
    Class<?> clazz = object.getClass();  
    if (!clazz.isAnnotationPresent(JsonSerializable.class)) {  
        throw new JSONException("The class "  
            + clazz.getSimpleName()  
            + " is not annotated with JsonSerializable");  
    }  
}
```

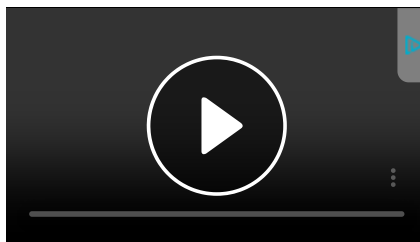
Em seguida, procuramos qualquer método com a anotação *@Init* e o executamos para inicializar os campos do nosso objeto:

```
private void initializeObject(Object object) throws Exception {  
    Class<?> clazz = object.getClass();  
    for (Method method : clazz.getDeclaredMethods()) {  
        if (method.isAnnotationPresent(Init.class)) {  
            method.setAccessible(true);  
            method.invoke(object);  
        }  
    }  
}
```

```
}
}
```

A chamada do método `.setAccessible ( true )` nos permite executar o método privado `initNames()` .

Após a inicialização, iteramos sobre os campos do nosso objeto, recuperamos a chave e o valor dos elementos JSON e os colocamos em um mapa. Então, criamos a string JSON a partir do mapa:



Patrocinado pela SABIC

Como estamos ajudando esses pezinhos a deixarem uma pegada...

Nossa colaboração com a Drylock e o Grupo Plastik está ajudando a criar uma solução para que as fraldas gerem menos resíduos, por meio da utilização dos polímeros circulares certificad...

Saber mais

```
private String getJsonString(Object object) throws Exception {
    Class<?> clazz = object.getClass();
    Map<String, String> jsonElementsMap = new HashMap<>();
    for (Field field : clazz.getDeclaredFields()) {
        field.setAccessible(true);
        if (field.isAnnotationPresent(JsonElement.class)) {
            jsonElementsMap.put(getKey(field), (String) field.get(object));
        }
    }

    String jsonString = jsonElementsMap.entrySet()
        .stream()
        .map(entry -> "\"" + entry.getKey() + "\":\""
            + entry.getValue() + "\"")
        .collect(Collectors.joining(", "));
    return "{" + jsonString + "}";
}
```

Novamente, usamos o campo `.setAccessible ( true )` porque os campos do objeto `Person` são privados.

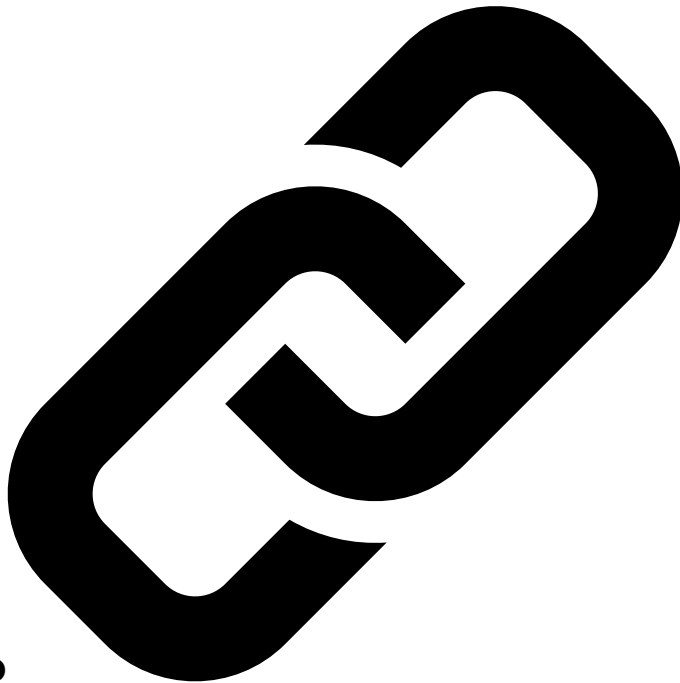
Nossa classe serializadora JSON combina todas as etapas acima:

```
public class ObjectToJsonConverter {
    public String convertToJson(Object object) throws JsonSerializationException {
        try {
            checkIfSerializable(object);
            initializeObject(object);
            return getJsonString(object);
        } catch (Exception e) {
            throw new JsonSerializationException(e.getMessage());
        }
    }
}
```

Por fim, executamos um teste de unidade para validar se nosso objeto foi serializado conforme definido por nossas anotações personalizadas:

```
@Test
public void givenObjectSerializedThenTrueReturned() throws JsonSerializationException {
    Person person = new Person("soufiane", "cheouati", "34");
    ObjectToJsonConverter serializer = new ObjectToJsonConverter();
    String jsonString = serializer.convertToJson(person);
    assertEquals(
```

```
{\ "personAge\ ": \"34\ ", \"firstName\ ": \"Soufiane\ ", \"lastName\ ": \"Cheouati\ " },  
jsonString);  
}
```



4. Conclusão

Neste artigo, aprendemos como criar diferentes tipos de anotações personalizadas. Em seguida, discutimos como usá-las para decorar nossos objetos. Por fim, vimos como processá-las usando a API Reflection do Java.

O código que dá suporte a este artigo está disponível no GitHub. Depois de fazer **login como Baeldung Pro Member**, comece a aprender e codificar no projeto.